

Questões:

1 - Considere as seguintes declarações de variáveis apresentadas e explique, conceitualmente, o que ocorre ao realizar, as declarações expostas em relação ao uso de memória.

```
void main() {  
    int a;  
    float b;  
    int w[10];  
    int m[5][5]  
}
```

Int é 4

Char 1

float 4 com ponto flutuante

double é 8

Struct soma do tamanho dos membros

Para as questões 02 e 03: Sobre algoritmos de Busca, considere o seguinte vetor:

int vetor[] = {50, 20, 25, 30, 35, 40, 45, 10};

2 - Busca Sequencial: Ao realizar a busca no referido vetor, quantas iterações serão necessárias até encontrar o valor 40 e o valor 99 (incluindo a iteração cujo valor é encontrado). São, respectivamente:

- a. 1 e 1 iteração
- b. 3 e 1 iterações
- c. 5 e 8 iterações
- d. 6 e 1 iterações
- e. 6 e 8 iterações

3 - Busca Sequencial Ordenada: Ao realizar a busca no referido vetor, quantas iterações serão necessárias até encontrar o valor 40 e o valor 10 (incluindo a iteração cujo valor é encontrado). São, respectivamente:

- a. 1 e 1 iteração
- b. 3 e 1 iterações

c. 5 e 1 iterações

d. 6 e 1 iterações

e. 8 e 1 iterações

4 - Explique o funcionamento da Busca Binária, resalte suas vantagens e desvantagens.

A busca binária trabalha com o conceito de dividir e conquistar, você faz um laço de repetição para correr o vetor e um if que dividi o vetor no meio se o elemento for maior do que o que você encontrou você olha a metade da frente (meio+1), else se for menor você olha a metade anterior (meio-1), else se for o que você está procurando você encontrou o valor. Vantagem é que você diminui a quantidade de iterações necessárias, desvantagem é necessário ter o vetor ordenado previamente.

5 - considere o seguinte trecho de código:

```
int main(){  
    int a = 44;  
    int b = 4;  
  
    int c = (--a + b++);  
  
    printf("Valor de C: %d", --c);  
}
```

- $c = 43 + 4 = 47$.
- Agora, c vale 47.

3. Executando o `printf("... %d", --c);`

- **--c (Pré-decremento):** O valor de c é reduzido **antes** de ser enviado para a função `printf`.
 - c era 47, passa a ser 46.

A saída deverá ser:

a. Valor de C: 47

b. Valor de C: 46

c. Valor de C: 45

d. Valor de C: 49

e. nenhuma das anteriores

6 - Tem-se os algoritmos abaixo sem erro de compilação/execução:

Algoritmo 1:

```
int sorted = 0;
while(sorted == 0){
    sorted = 1;
    for(int i = 0; i < LENGTH - 1; i++){
        if(v[i] > v[i + 1]) {
            int temp = v[i];
            v[i] = v[i + 1];
            v[i + 1] = temp;

            sorted = 0;
        }
    }
}
```

Algoritmo 2:

```
for(int i = 0; i < LENGTH; i++){
    for(int j = 0; j < LENGTH; j++){
        if(v[j] > v[j + 1]){
            int aux = v[j];
            v[j] = v[j + 1];
            v[j + 1] = aux;
        }
    }
}
```

a. O que os algoritmos fazem? Ambos os algoritmos realizam a **ordenação** de um vetor de forma crescente utilizando o método **Bubble Sort** (ou Ordenação por Bolha). Eles comparam pares de elementos vizinhos e os trocam de posição caso o anterior seja maior que o próximo.

b. Explique se há diferença de funcionamento de ambos, se houver, detalhe qual. Sim. Embora ambos sejam Bubble Sort, o Algoritmo 1 é uma versão otimizada e o Algoritmo 2 é uma versão simples vai usando a variável `sorted` para ordenar. O Algoritmo 1 usa uma variável chamada `sorted` (como um sinalizador ou *flag*). Ele começa cada volta assumindo que o vetor já está pronto (`sorted = 1`). Se ele fizer qualquer troca, ele muda para 0. Se ele terminar uma volta inteira sem trocar nada, o `while` entende que o vetor está ordenado e **para o programa mais cedo**, economizando iterações. Já o segundo vai percorrer o vetor o número máximo de vezes, mesmo que os números já estejam em ordem logo na primeira tentativa. Ele gasta mais processamento desnecessariamente.

O Algoritmo 1 tem um "sensor de prontidão" (`sorted`), enquanto o Algoritmo 2 trabalha no automático até o fim, independente do resultado. Se o professor perguntar sobre erros, note que o **Algoritmo 2** tem um perigo: o laço vai até `j < LENGTH`. Quando chegar no último elemento, ao tentar comparar `v[j]` com `v[j+1]`, ele vai tentar acessar uma posição que **não existe** (fora do vetor), o que pode travar o programa. O **Algoritmo 1** evita isso usando `LENGTH - 1`.

7 - Sobre estruturas definidas pelo usuário:

Cadastro : NOME, ENDEREÇO, SALÁRIO, IDENTIDADE, CPF, ESTADO CIVIL, TELEFONE, IDADE, SEXO

O ENDEREÇO é composto de: RUA, BAIRRO, CIDADE, ESTADO, CEP

a. explique qual sua finalidade em um exemplo real de aplicação; é permitir a criação de **novos tipos de dados** que agrupam variáveis de tipos diferentes sob um mesmo nome. Isso facilita a organização e a manipulação de informações que, logicamente, pertencem a um mesmo objeto. b. crie as structs que melhor representam as relações. Um exemplo real seria o cadastro de cliente de uma loja, com a struct o sistema já acessa o bloco de informações limpando o nosso código.

b. Crie as structs que melhor representam as relações:

```
struct Endereco {  
    char rua[100];  
    char bairro[50];  
    char cidade[50];  
    char estado[2];
```

```
char cep[10];

};

struct Cadastro {

char nome[100];
struct Endereco endereco;
float salario;
char identidade[20];
char cpf[15];
char estadoCivil[20];
char telefone[20];
int idade;
char sexo[2];

};
```

8. Considere a seguinte implementação de funções:

```
void somaA(int a, int b, int *c) {

    *c = a + b; (aqui eu armazeno o conteúdo)

}
```

```
int somaB(int a, int b) {

    return a + b;

}
```

Responda:

a. Para invocar ambas as funções a partir do método main e/ou de outras funções, qual a implementação correta o uso de ambas as funções (represente exemplo de como funções em termos de código). Faça o código que permita mostrar o resultado no console;

```
Main{

int resA, resB;

somaA(x,y,&resA);

resB= soma(x,y);

printf("Resultado SomaA: %d", resA);

printf("Resultado SomaA: %d", resB);
```

b. qual a diferença entre ambas funções?

A primeira função é do tipo void e os parâmetros foram passados por referência, já na segunda função os parâmetros foram passados por valor e tem retorno pois é uma função do tipo int e tem retorno.

9 - Analise o código abaixo:

```
struct geoloc {  
    float lat;  
    float lon;  
};  
  
typedef struct geoloc GeolocPoint;  
  
int main(){  
    GeolocPoint g1 = {1.9838, 1.3030};  
  
    GeolocPoint *p = &g1;  
  
    GeolocPoint **y = &p;  
  
    printf("A: %f", p->lat);  
    printf("\nB: %f", (*y)->lat);  
  
    printf("\n\nC: %f", g1.lat);  
    printf("D: %f", p->lat);  
    printf("\nE: %f", (*y)->lat)  
}
```

Responda:

- a. Quais serão as saídas para A, B, C, D e E?
- **A:** 1.983800
 - **B:** 1.983800
 - **C:** 1.983800
 - **D:** 1.983800

- • E: 1.983800

vai ser a latitude

b. Explique o conceito de ponteiros utilizados no código.

O ponteiro aponta um endereço de memória no A eu aponto o endereço de memória do p que recebeu a struct g1 e especifico que quero exibir o bloco latitude, no B eu aponto o ponteiro y que recebeu o ponteiro p que aponta o endereço de memória da struct g1 e com o (*y) mostro o ponteiro e com -> mostro qual bloco quero acessar. No ponto C eu uso a estrutura de mostrar na variável sem usar o ponteiro e mostro que quero exibir o dado da lat.

10 - Analise o código abaixo:

```
int main(void){  
    int i = 10;  
    int *p;  
  
    p = &i;  
    *p = *p * 2;  
  
    printf("Valor de i: %d", i);  
    printf("Valor de p: %d", *p);  
}
```

a. Quais serão as saídas deverão ser as saídas de i e p? 20 e 20

b. Explique o que ocorreu no código.

Note que no `printf`, o código utiliza `*p` (o valor apontado) e não apenas `p` (o endereço). Além disso, como `p` aponta diretamente para o espaço de memória de `i`, qualquer alteração feita através do ponteiro reflete no valor da variável original.